

Tech Stack Justification Document

SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 **Status:** Final Recommendation for Architecture Phase **Based On:** Product Vision v1.0, PRD v1.0, SRS v1.0

1. Purpose

This document evaluates the major technology choices required to build SentinelCut and provides a final, justified recommendation for each layer of the stack: speech recognition, profanity detection architecture, audio/video processing, backend framework, frontend framework, database, storage, authentication, and deployment. Every recommendation is made against the specific constraints established in the SRS: **free/open-source-first, CPU-feasible, Hindi/Hinglish-capable, and buildable within an academic timeline.**

2. Evaluation Criteria (Applied Throughout)

Criterion	Why It Matters for This Project
Accuracy (esp. Hindi/Hinglish)	Core product differentiator per Product Vision
Offline/CPU feasibility	No guaranteed GPU budget; free-tier academic constraint
Cost	Must be free or near-free for MVP
Ease of implementation	Small team, limited timeline
Community support/documentation	Reduces implementation risk
Scalability path	Should not be a dead end if the project continues post-graduation

3. Speech Recognition (ASR) Engine

3.1 Options Compared

Option	Accuracy	Hindi/Hinglish Support	Offline	Speed	GPU Need	Cost	Ease of Use
OpenAI Whisper (open-source)	High	Good (multilingual training incl. Hindi)	Yes	Moderate (slower on CPU)	Optional (faster with GPU)	Free	Easy (pip install, well documented)
WhisperX	High + word-level timestamps	Good (built on Whisper)	Yes	Faster than base Whisper (batched inference + forced alignment)	Optional	Free	Moderate (extra alignment dependencies)
Faster-Whisper (CTranslate2 reimplementation)	Same as Whisper, faster	Good	Yes	Significantly faster on CPU than original Whisper	Optional	Free	Easy
Vosk	Moderate	Weaker on code-mixed/Hinglish	Yes	Fast, lightweight	No	Free	Easy but lower accuracy
NVIDIA NeMo	High	Requires custom Hindi model training/fine-tuning	Yes	Fast on GPU	Strongly GPU-recommended	Free (compute cost)	Complex, steep learning curve
Google Speech-to-Text API	Very High	Good Hindi support	No (cloud only)	Fast	N/A (cloud)	Paid beyond free tier	Very easy (API call)
Azure Speech	Very High	Good Hindi support	No (cloud only)	Fast	N/A (cloud)	Paid beyond free tier	Very easy (API call)
Deepgram	High	Limited Hindi/Hinglish maturity vs. Whisper family	No	Very fast	N/A (cloud)	Paid beyond free tier	Very easy (API call)
AssemblyAI	High	Limited Hindi/Hinglish maturity	No	Fast	N/A (cloud)	Paid beyond free tier	Very easy (API call)

3.2 Key Trade-offs

- **Cloud APIs (Google, Azure, Deepgram, AssemblyAI)** offer the best raw accuracy and zero setup effort, but they violate two hard constraints: they require ongoing per-minute billing beyond limited free tiers, and they are not viable for an "offline-capable, cost-free academic MVP." They also create a hard external dependency that risks demo failure if free-tier quotas are exhausted.
- **Vosk** is attractive for being lightweight and fully offline, but its Hindi and especially Hinglish/code-mixed accuracy is noticeably weaker than Whisper-family models, which directly undermines the project's core differentiator.
- **NVIDIA NeMo** can achieve excellent results but requires either a pretrained Hindi checkpoint of uncertain quality or custom fine-tuning — both of which add significant time/compute risk that is disproportionate for an academic timeline.
- **The Whisper family (Whisper / WhisperX / Faster-Whisper)** consistently offers the best balance: strong multilingual training (including Hindi and reasonable handling of code-switched speech), fully offline/open-source (no API cost), and an active community with abundant implementation resources.

3.3 Recommendation: Faster-Whisper as the primary ASR engine, with WhisperX-style forced alignment for word-level timestamps

Why:

1. **Accuracy:** Inherits Whisper's multilingual training, which performs meaningfully better on Hindi and mixed-language speech than Vosk or untrained NeMo checkpoints.
2. **Speed on CPU:** Faster-Whisper (built on CTranslate2) is significantly faster than vanilla Whisper on CPU-only infrastructure — critical given the "no guaranteed GPU" constraint from the SRS.
3. **Offline & free:** No per-minute API cost, no external dependency risk during a live demo.
4. **Word-level timestamps:** The Timeline Editor (SRS-FR-TLE) requires precise timestamps per detected word. Whisper's native segment-level timestamps are too coarse; incorporating a **forced-alignment step** (the technique WhisperX popularizes, e.g., via wav2vec2-based alignment or Faster-Whisper's built-in word-timestamp option) is necessary to meet FR-TLE-01/02 and the <100ms sync requirement in the Product Vision.
5. **GPU as upgrade path, not dependency:** If GPU becomes available (e.g., free Colab/Kaggle credits during development, or paid infra post-graduation), the same model scales up in speed without an architecture change.

Fallback design note: The ASR engine should sit behind an internal `SpeechRecognitionService` interface (per Repository/Service pattern in Coding Standards) so it can be swapped for a cloud API (e.g., Google Speech-to-Text) in a future paid tier without touching the rest of the pipeline.

4. Profanity Detection Architecture

4.1 Options Compared

Approach	Accuracy on Slang/Variants	Multilingual Handling	Explainability	Implementation Effort	Data Requirement
Pure Dictionary Matching	Poor (misses variants, spelling tricks)	Requires manual per-language dictionaries	High	Low	Curated word lists only
Regex-based matching	Moderate (handles some patterns like repeated letters)	Same as dictionary, plus pattern rules	High	Low-Medium	Curated word lists + patterns
Context-aware Transformer (BERT/roBERTa)	High (understands context, disambiguates)	Requires multilingual or fine-tuned model	Medium	Medium-High	Labeled training data (limited availability for Hindi/Hinglish profanity)
IndicBERT / MuRIL (India-focused multilingual BERT variants)	High for Indian languages	Purpose-built for Hindi/Indic + code-mixed text	Medium	Medium	Some public Indic NLP resources available; profanity-specific labels still limited
XLM-R (cross-lingual roBERTa)	High, broad multilingual	Strong general multilingual, less India-specific tuning than IndicBERT/MuRIL	Medium	Medium-High	Similar data scarcity issue
LLM Classification (zero/few-shot via an LLM)	High, flexible, handles novel slang via reasoning	Strong multilingual understanding out of the box	Low (harder to audit exact rule)	Low (prompt-based) but ongoing inference cost	No training data required, but adds per-request cost/latency
Sentence Transformers (embedding similarity to profanity examples)	Moderate-High	Depends on base multilingual embedding model	Medium	Medium	Needs a seed set of example profane phrases per language

4.2 Key Trade-offs

- **Pure dictionary/regex** is fast, explainable, and requires zero training data — but on its own it cannot reliably catch creative obfuscation (e.g., "fuuuuck," "m@darchod," novel slang), which the Product Vision explicitly requires the system to handle.
- **Pure transformer/LLM classification** is powerful but has two problems for this project: (1) labeled Hindi/Hinglish profanity datasets are scarce, making fine-tuning a full context-aware classifier from scratch a significant undertaking within an academic timeline, and (2) LLM-based classification for every transcript segment adds latency and, if cloud-hosted, cost — conflicting with the offline/free-tier constraint.
- **IndicBERT/MuRIL** are the most relevant pretrained multilingual models for this specific problem (India-focused, code-mixed aware) but still benefit from being combined with a rule-based layer rather than used in isolation, given limited profanity-specific labeled data.

4.3 Recommendation: Hybrid Architecture — Normalization + Dictionary/Regex Layer + Lightweight Context Model (fine-tuned/adapted multilingual embedding or IndicBERT-based classifier) as a confidence booster

Why a hybrid, not a single method:

This directly answers the Product Vision requirement to "explain whether the model should be Rule-Based, Machine Learning, Deep Learning, or Hybrid" — **Hybrid is the correct choice** for this project specifically, because:

1. **No single approach satisfies both requirements simultaneously:** catching obvious/known profanity with high explainability (favors rule-based) *and* catching context-dependent, disguised, or novel slang (favors ML/DL). A hybrid captures both strengths.
2. **Data scarcity is the binding constraint.** A pure deep-learning classifier needs substantial labeled Hindi/Hinglish profanity data that does not readily exist; relying on it alone is high-risk for an academic timeline. A hybrid lets the dictionary/rule layer carry most of the load on day one while the ML layer is incrementally improved.
3. **Explainability matters for user trust.** Because every detection is shown to the user in the Timeline Editor with a confidence score and source tag (SRS-FR-PDE-05/07), a hybrid system can clearly label "dictionary match" vs. "model-inferred," which a black-box-only classifier cannot do as cleanly.

Recommended pipeline layers (in order):

1. **Text Normalization Layer** — lowercasing, Unicode normalization (for Devanagari), repeated-character collapsing ("fuuuuck" → "fuck"), leetspeak/symbol substitution mapping ("m@darchod" → "madarchod").
2. **Dictionary + Regex Matching Layer** — curated multilingual profanity dictionary (English + Hindi + common Hinglish/abbreviated slang such as "bc," "mc," "bkl") matched against normalized text; assigns high-confidence detections instantly.
3. **Context-Aware Scoring Layer** — a lightweight multilingual sentence embedding model (e.g., a MuRIL/IndicBERT-based or general multilingual sentence-

transformer model) computes similarity between each transcript segment and a seed bank of profane example phrases, used to (a) catch terms the dictionary missed and (b) adjust confidence for borderline/context-dependent cases.

4. **Confidence Aggregation** — final confidence score for each detection instance is a combination of match-source reliability (dictionary > regex > embedding-similarity) surfaced to the user per SRS-FR-PDE-05.

This design also directly satisfies the Product Vision's requirement for "**support for future learning**" — the embedding-similarity layer can be improved over time simply by expanding the seed example bank, without retraining a full classifier from scratch, and the dictionary is designed to be admin-editable at runtime (SRS-FR-PDE-08).

5. Audio & Video Processing Libraries

5.1 Options Compared

Library	Purpose	Strengths	Weaknesses
FFmpeg	Industry-standard audio/video processing	Extremely fast, handles virtually every codec/container, precise frame/timestamp control, muxing	Command-line/subprocess-based; steeper learning curve than Python-native libraries
MoviePy	Python video editing wrapper (uses FFmpeg internally)	Pythonic API, good for prototyping edits	Slower for large files, less precise frame-accurate control, adds an abstraction layer over FFmpeg that can obscure performance issues
PyDub	Python audio manipulation	Simple API for audio slicing/overlaying (ideal for beep/mute insertion)	Audio-only; relies on FFmpeg/libav under the hood anyway; not suited for video muxing
Librosa	Audio analysis (not editing)	Excellent for audio feature extraction/analysis (e.g., silence detection, spectral features)	Not designed for writing/exporting production audio; analysis-focused, not an editing tool
Torchaudio	PyTorch-native audio I/O and transforms	Good if audio processing needs to integrate directly with a PyTorch ML pipeline (e.g., feeding ASR models)	Not designed for general-purpose muxing/export workflows

5.2 Recommendation: FFmpeg as the core processing engine, with PyDub for audio segment manipulation (beep/mute/custom-sound insertion) at the Python service layer, and Librosa reserved for any future audio-analysis features (e.g., speaker diarization)

Why:

- **FFmpeg** is non-negotiable for the final mux/export step: it is the only option here with the precision, format coverage, and speed required to satisfy SRS-NFR-PERF-01 ($\leq 2\times$ real-time processing) and the Product Vision's requirement to preserve video quality, frame rate, resolution, and metadata.
- **PyDub** provides the most direct, readable API for the specific operation of "replace a time-bounded audio segment with a beep tone / silence / custom clip," which is the exact operation the Replacement module (SRS-FR-REPL) needs — and PyDub itself shells out to FFmpeg, so there is no redundant dependency.
- **MoviePy** is deliberately not used as the primary engine: its abstraction over FFmpeg tends to be slower and less precise for the frame-accurate, timestamp-critical edits this project requires, though it remains a reasonable option for quick prototyping during early development.
- **Librosa** and **TorchAudio** are not part of the core censorship pipeline but are the correct tools to reach for if/when the project adds analysis-heavy future features like speaker diarization or emotion detection (see Future Scope).

Processing flow: Extract audio (FFmpeg) → run ASR + detection on extracted audio → apply beep/mute/custom-sound edits to the audio track (PyDub) → re-mux edited audio back into the original video container (FFmpeg), preserving original video stream untouched to guarantee resolution/frame-rate/metadata preservation.

6. Backend Framework

6.1 Options Compared

Framework	Language	Async Support	ML/AI Integration	Ecosystem Fit	Learning Curve
FastAPI	Python	Native async, high performance	Excellent — same language as the AI pipeline (Whisper, PyTorch, etc.)	Auto-generated OpenAPI docs, strong typing via Pydantic	Low-Medium
Flask	Python	Limited (needs extensions for async)	Good, same-language benefit	Simple, minimal, very flexible	Low
Django	Python	Limited native async (improving)	Good, same-language benefit	Batteries-included (admin panel, ORM) but heavier than needed	Medium
Express	Node.js/JavaScript	Native async	Requires cross-language calls to Python AI services (extra complexity)	Huge ecosystem	Low-Medium
NestJS	Node.js/TypeScript	Native async	Same cross-language issue as Express	Structured, enterprise-style architecture	Medium-High
Go	Go	Native, very high performance	Requires cross-language calls to Python AI services	Excellent performance, smaller AI/ML ecosystem	Medium-High

6.2 Recommendation: FastAPI

Why:

1. **Same-language pipeline.** The entire AI stack (Whisper/Faster-Whisper, the profanity detection layer, PyDub) is Python. Using FastAPI avoids the significant added complexity of cross-language service calls (e.g., a Node.js backend calling out to a separate Python microservice) that Express, NestJS, or Go would require — a meaningful simplification for a small academic team.
2. **Async performance.** FastAPI's native async support is well suited to a system with a mix of I/O-bound work (file upload/download, DB queries) and long-running background jobs (AI processing), which need to be offloaded to a queue rather than block the request thread.
3. **Automatic API documentation** (OpenAPI/Swagger) directly supports the API Design and API Documentation deliverables required later in the project.
4. **Pydantic-based validation** enforces clean request/response schemas, aligning with SRS-NFR-SEC-03 (input validation/sanitization).
5. Compared to Flask, FastAPI provides async and validation out of the box rather than requiring extensions; compared to Django, FastAPI avoids unnecessary weight (a full ORM/admin/templating stack) that is largely redundant given the project already plans a separate frontend and a lightweight custom admin panel.

7. Frontend Framework

7.1 Options Compared

Framework	Rendering	SEO (Landing Page)	Component Ecosystem	Fit for Rich Interactive UI (Timeline Editor)	Learning Curve
Next.js (React-based)	SSR + CSR hybrid	Strong (SSR for landing/marketing pages)	Very large (shares React ecosystem)	Excellent, with mature libraries for waveform/timeline UIs	Medium
React (CRA/Vite, no SSR)	CSR only	Weaker without extra SSR setup	Very large	Excellent	Medium
Vue	SSR possible (Nuxt)	Strong with Nuxt	Large, slightly smaller than React	Good, fewer ready-made timeline/waveform libraries than React	Low-Medium
Angular	SSR possible (Angular Universal)	Strong with setup	Large but more opinionated/heavier	Good but higher boilerplate for custom interactive components	High

7.2 Recommendation: Next.js

Why:

1. **Hybrid rendering fits the product's dual nature:** SentinelCut needs a marketing/landing page (benefits from SSR for load speed and shareability) *and* a highly interactive authenticated app (Dashboard, Timeline Editor) that behaves like a client-rendered SPA once logged in. Next.js supports both patterns natively in one framework.
2. **Largest ecosystem for the hardest UI piece.** The Timeline Editor is the most technically demanding UI component in the product (synced transcript + audio/video playback + waveform-like highlighting). React's ecosystem has the most mature libraries and community examples for building this kind of media-timeline interface, which reduces implementation risk more than Vue or Angular.
3. **Tailwind CSS integration** (mandated by the SRS/PRD) is first-class and extremely well documented in the Next.js ecosystem.
4. **Deployment simplicity:** Next.js apps deploy seamlessly to free-tier hosting (e.g., Vercel), directly supporting the "recommend free services for a college project" requirement.

8. Database

8.1 Options Compared

Database	Data Model Fit	Free Tier Availability	Query Flexibility	Ops Overhead
PostgreSQL	Excellent for structured, relational data (users, files, detections, jobs) with JSON columns available for flexible metadata	Good (via Supabase/Neon/Railway free tiers)	Full SQL + JSONB flexibility	Low with managed hosting
MongoDB	Good for flexible/nested documents (e.g., detection lists) but weaker for strictly relational data (users↔files↔credits↔roles)	Good (MongoDB Atlas free tier)	Flexible but weaker joins/relational integrity	Low with managed hosting
Supabase (Postgres + BaaS layer)	Same underlying model as PostgreSQL, plus built-in auth/storage/realtime	Generous free tier	Full SQL	Very low (managed)
Firebase (Firestore)	Document-based, real-time sync strength	Generous free tier	Limited complex querying/joins	Very low (managed)
SQLite	Simple relational, file-based	N/A (local file, no hosting needed)	Full SQL but not built for concurrent multi-user production load	Very low, but not production-appropriate for a multi-user SaaS

8.2 Recommendation: PostgreSQL (via Supabase-managed hosting)

Why:

- 1. **Data shape is inherently relational.** Users, files, detection instances, credits, roles, and admin logs have clear relationships (foreign keys, joins) that map cleanly onto a relational schema — the Database Design document (next in the documentation pipeline) will define entities such as `users`, `files`, `detections`, `presets`, `jobs`, and `admin_logs`, which is a textbook relational use case.
- 2. **JSONB support** in PostgreSQL still allows flexible storage where needed (e.g., storing the full JSON detection metadata blob per file) without giving up relational integrity elsewhere — getting the best of both the SQL and document-store worlds.
- 3. **Supabase specifically** is recommended as the hosting layer because it bundles Postgres with authentication and storage in one free-tier-friendly service, directly simplifying the "Deployment: recommend free services for college project" requirement (see Section 10), while still being "just Postgres" underneath — avoiding vendor lock-in risk associated with Firebase's proprietary Firestore query model.
- 4. Compared to MongoDB/Firestore, Postgres avoids the well-known difficulty of expressing relational analytics queries (e.g., admin analytics: "average processing time per user per week") in a document store.
- 5. SQLite is excluded for production/demo hosting despite its simplicity, since it is not designed for the concurrent multi-user access a deployed SaaS demo requires — though it remains useful for local unit testing.

9. Storage

9.1 Options Compared

Storage Option	Media File Fit	Free Tier	Integration Effort
Cloudinary	Strong for images/video with built-in transformations	Limited free tier for video-heavy use	Easy SDKs
AWS S3	Industry standard for large media files	Free tier limited in duration (12 months) and volume	Moderate (IAM setup)
Supabase Storage	Good for general file storage, integrates directly with Supabase Auth/DB (signed URLs, row-level security)	Generous free tier	Very easy, same ecosystem as chosen DB/Auth
Local Storage (server disk)	Simplest for a pure academic demo	Free (no cloud dependency)	Very easy but not scalable, risk of data loss on redeploy, not production-shaped

9.2 Recommendation: Supabase Storage (primary), with Local Storage as a fallback for early local development/testing only

Why:

1. **Ecosystem consistency.** Since PostgreSQL is already hosted via Supabase, using Supabase Storage for uploaded/processed media files means authentication, database, and storage share one consistent permissions model (Row-Level Security policies can directly govern who can access which files) — reducing integration complexity versus stitching together a separate AWS S3 account with its own IAM configuration.
 2. **Free-tier friendliness** directly satisfies the constraint of using free services for a college project, while still being a "real" production-grade object storage service (not a throwaway local-disk hack), keeping the architecture credible to both academic evaluators and future investors.
 3. **AWS S3** remains the recommended upgrade path if the project scales post-graduation and needs finer-grained storage-class/cost optimization, but it is not necessary for MVP.
 4. **Cloudinary** is deprioritized here because its core value proposition (image/video transformation pipeline) overlaps with, rather than adds to, the custom FFmpeg-based processing pipeline this project already needs to build — it would be a partially redundant dependency.
-

10. Authentication

10.1 Options Compared

Option	Custom Auth	OAuth Support	Integration Effort	Fit
JWT (self-managed)	Full control	Requires manual OAuth wiring	Higher	Good if team wants full control, more code to maintain
OAuth (standalone, e.g., Google directly)	N/A	Yes	Moderate	Needed regardless, for Google Login requirement
NextAuth (Auth.js)	Good	Excellent, many providers pre-built	Low	Strong fit specifically because the frontend is Next.js
Clerk	Good, hosted UI components	Excellent	Very low	Strong DX but adds a third-party auth vendor dependency/cost at scale
Supabase Auth	Good	Good (Google OAuth supported)	Low, same ecosystem as DB/Storage	Strong fit given Supabase is already the DB/Storage choice

10.2 Recommendation: Supabase Auth (JWT-based under the hood) for the primary user/session management, satisfying both the "JWT" and "OAuth" requirements simultaneously

Why:

1. **Single-vendor coherence.** Since Postgres (DB) and Storage are both hosted on Supabase, using Supabase Auth as well means a single set of credentials and a single Row-Level Security model governs data access across all three layers — this is a meaningfully simpler architecture for a small team than integrating NextAuth/Clerk as a fourth separate service on top.
2. **Satisfies FR-AUTH requirements directly:** Supabase Auth natively issues JWTs (SRS-FR-AUTH-06) and supports Google OAuth (SRS-FR-AUTH-02) and email/password with reset flows (SRS-FR-AUTH-01/03) out of the box.
3. **Guest Mode** (SRS-FR-AUTH-04) is implemented as an application-level pattern (an anonymous/temporary record with a scoped, time-limited token) layered on top of Supabase Auth rather than requiring a separate system.
4. **NextAuth** remains a very reasonable alternative if the team later decouples from Supabase for auth specifically (e.g., adds more OAuth providers), and **Clerk** is noted as the best pure "developer experience" option but is deprioritized here to

avoid adding a paid third-party vendor dependency beyond what's needed for MVP.

11. Deployment Platforms (Free-Tier Recommendation for Academic Project)

Layer	Recommended Service	Why
Frontend (Next.js)	Vercel (free tier)	Purpose-built for Next.js, zero-config deploys, generous free tier, automatic HTTPS
Backend (FastAPI)	Render or Railway (free/hobby tier)	Simple Docker-based deploys for Python services, supports background workers for the processing queue
AI Processing Workers	Same host as backend (Render/Railway background worker) for MVP; Google Colab/Kaggle free GPU credits usable during development/testing for heavier experimentation	Keeps infra simple for MVP; avoids managing a separate GPU cluster
Database	Supabase (managed Postgres, free tier)	Already justified in Section 8
Storage	Supabase Storage (free tier)	Already justified in Section 9
Queue/Background Jobs	Redis (free tier via Upstash) + a Python task queue (e.g., Celery or RQ)	Needed to satisfy SRS-FR-DASH-05 (live processing queue) and NFR-PERF-04 (concurrent job handling) without blocking the API
Email (password reset)	A transactional email free-tier provider (e.g., Resend or similar)	Needed for SRS-FR-AUTH-03

Note: Exact free-tier limits and specific provider names change frequently; the pattern recommended here (SSR-frontend host + containerized Python backend host + managed Postgres/Storage/Auth + lightweight Redis-backed queue) is the architecturally important decision, and the specific vendor can be swapped without redesigning the system if a given free tier changes or is discontinued.

12. Consolidated Tech Stack Summary

Layer	Final Recommendation
ASR Engine	Faster-Whisper (+ forced alignment for word-level timestamps)
Profanity Detection	Hybrid: Normalization → Dictionary/Regex → Multilingual embedding/IndicBERT-style context scoring
Audio/Video Processing	FFmpeg (core), PyDub (segment-level beep/mute/custom-sound edits)
Backend Framework	FastAPI (Python)
Frontend Framework	Next.js (React, Tailwind CSS)
Database	PostgreSQL (via Supabase)
Storage	Supabase Storage
Authentication	Supabase Auth (JWT + Google OAuth)
Queue/Background Jobs	Redis + Celery/RQ
Deployment	Vercel (frontend) + Render/Railway (backend & workers) + Supabase (DB/Storage/Auth)

This stack is intentionally **Python-unified on the AI/backend side** (simplifying integration between FastAPI and the Whisper/PyDub/FFmpeg pipeline), **Supabase-unified on the data side** (simplifying Auth/DB/Storage integration and free-tier hosting), and **React/Next.js on the frontend** (best-fit ecosystem for the Timeline Editor's interactive requirements) — directly satisfying every constraint set out in the SRS: free-tier feasibility, CPU-first operation, Hindi/Hinglish capability, and buildability within an academic timeline, while remaining a credible, scalable foundation if the project continues beyond graduation.

13. Next Document

Document 5: System Architecture — translating this tech stack into a complete architecture with component diagrams, sequence diagrams, data flow diagrams, and deployment diagrams, building directly on the recommendations above.

End of Tech Stack Justification Document